

ДОДАТОК В

Специфікація програмного забезпечення

Приватний месенджер для локальної мережі

Специфікація Програмного Забезпечення

1.0

19.04.2026

Тищенко Максим Владиславович

ЗМІСТ

1 Вступ.....	4
1.1 Огляд продукту.....	4
1.2 Мета	4
1.3 Межі.....	5
1.4 Посилання	5
1.5 Означення та аббревіатури.....	6
2 Загальний опис	7
2.1 Перспективи продукту.....	7
2.2 Функції продукту	7
2.3 Характеристики користувачів.....	8
2.4 Загальні обмеження	8
2.5 Припущення й залежності.....	9
3 Конкретні вимоги	10
3.1 Вимоги до зовнішніх інтерфейсів	10
3.1.1 Інтерфейс користувача	10
3.1.2 Апаратний інтерфейс.....	10
3.1.3 Програмний інтерфейс	11
3.1.4 Комунікаційний протокол.....	11
3.1.5 Обмеження пам'яті	11
3.1.6 Операції.....	12
3.1.7 Функції продукту	12
3.1.8 Припущення й залежності.....	13
3.2 Властивості програмного продукту	13
3.3 Атрибути програмного продукту	14
3.3.1 Надійність	14
3.3.2 Доступність.....	14
3.3.3 Безпека	14
3.3.4 Супроводжуваність.....	14
3.3.5 Переносимість	15

3.4 Вимоги до збереження даних (Бази даних)	15
3.5 Інші вимоги	16

1 ВСТУП

1.1 Огляд продукту

MTMessenger є програмною системою клієнт-серверного обміну повідомленнями у реальному часі, призначеною для багатокористувацької комунікації в межах локальних мереж. Користувач може підключатися до центрального сервера, використовуючи унікальний ідентифікатор (нікнейм), та вести листування у загальному (глобальному) чаті або у приватних діалогах з іншими користувачами. Система автоматично синхронізує повідомлення між клієнтами та надає розширені інструменти керування контентом: множинне хронологічне закріплення повідомлень у діалогах, ручне Drag-and-Drop сортування закріплених чатів, комбінований пошук за іменами та текстом, а також керування сповіщеннями (Mute). З метою забезпечення інформаційного суверенітету користувача, система підтримує функцію синхронного повного очищення історії на обох пристроях співрозмовників та інтерактивний модуль локальної архівації всієї сесії у текстовий формат під час виходу з програми. Програмний продукт реалізовано як десктопний застосунок на базі платформи .NET із використанням технології WPF для графічного інтерфейсу та мережевих сокетів TCP/IP для передавання даних.

1.2 Мета

Цей документ описує вимоги до програмної системи MTMessenger та встановлює угоди між розробником і замовником щодо функцій, які система виконуватиме, і функцій, які нею не передбачені. Метою проєкту є розробка надійного та швидкого локального месенджера, який автоматизує та спрощує процеси обміну текстовою інформацією, надаючи користувачам гнучкі інструменти для сортування пріоритетних чатів та забезпечення приватності власних даних без їх примусового збереження на сторонніх хмарних серверах.

1.3 Межі

Система охоплює повний цикл текстової комунікації між клієнтами та сервером. Користувач може авторизуватися за нікнеймом, обмінюватися текстовими повідомленнями, використовувати пошук (глобальний і локальний), закріплювати та відкріплювати чати в лівій панелі з можливістю зміни їхнього порядку мишею. У межах конкретного чату користувач може закріплювати кілька повідомлень, перемикатися між ними, вимикати звукові сповіщення та очищувати історію локально або глобально (для всіх учасників). Користувач також може створити локальну резервну копію історії чатів у форматі .txt. Система не підтримує передачу мультимедійних файлів (зображень, аудіо, відео) та документів. Програмний продукт не включає функціонал голосових чи відеодзвінків. Система орієнтована на розгортання у локальних мережах (LAN) і не має вбудованих механізмів наскрізного криптографічного шифрування (End-to-End Encryption) або інтеграції із зовнішніми базами даних для довготривалого зберігання профілів (сервер зберігає дані сесій лише в оперативній пам'яті до моменту перезавантаження).

1.4 Посилання

- Платформа .NET. URL: <https://dotnet.microsoft.com/>
- Windows Presentation Foundation (WPF) Documentation. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>
- Мова програмування C#. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/> – System.Net.Sockets Namespace (TCP/IP). URL: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets>
- System.Text.Json Serialization. URL: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json-overview>
- XAML (Extensible Application Markup Language) Overview. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/advanced/xaml-overview>

1.5 Означення та аббревіатури

- WPF (Windows Presentation Foundation) – графічна підсистема у складі .NET для створення візуальних інтерфейсів десктопних застосунків під ОС Windows.
- XAML (Extensible Application Markup Language) – декларативна мова розмітки, що використовується для ініціалізації структури об'єктів та побудови користувацьких інтерфейсів у WPF.
- TCP/IP (Transmission Control Protocol / Internet Protocol) – набір базових мережових протоколів передавання даних, що гарантує цілісність та правильну послідовність доставлення пакетів.
- Socket (Сокет) – програмний інтерфейс для забезпечення інформаційного обміну між процесами в мережі.
- JSON (JavaScript Object Notation) – текстовий формат обміну даними, що використовується в системі для серіалізації мережових пакетів (MessagePacket).
- Drag-and-Drop – спосіб взаємодії з графічним інтерфейсом, що полягає у захопленні візуального елемента (наприклад, чату) мишею та його переміщенні в інше місце.
- Mute – функція вимкнення візуальних та звукових сповіщень для обраного чату.
- Pin – функція жорсткого закріплення елемента (чату або повідомлення) у верхній частині списку поза хронологічним порядком.
- UI (User Interface) – графічний інтерфейс користувача.
- UX (User Experience) – досвід користувача, логіка та зручність його взаємодії з програмою.

2 ЗАГАЛЬНИЙ ОПИС

2.1 Перспективи продукту

MTMessenger є самостійною клієнт-серверною системою, що функціонує незалежно від глобальних хмарних платформ і призначена для організації комунікації у закритих локальних мережах. Система вирішує практичну проблему збереження інформаційного суверенітету та конфіденційності: сучасні комерційні месенджери зберігають усю історію листування на сторонніх серверах та часто не дозволяють користувачам безперешкодно експортувати свої дані чи гарантовано видаляти їх синхронно для всіх учасників бесіди. MTMessenger надає користувачам повний контроль над своїми даними, забезпечуючи миттєвий текстовий обмін та розширені можливості локального архівування сесій.

Система має значний потенціал для подальшого розширення у напрямках: інтеграції реляційної бази даних на стороні сервера для довготривалого збереження історії (офлайн-повідомлення), додавання підтримки передачі мультимедійних файлів, впровадження алгоритмів наскрізного шифрування (E2EE), а також розширення рольової моделі для створення груп з адміністраторами та модераторами.

2.2 Функції продукту

Система забезпечує такі основні функції:

- підключення до сервера та автентифікація користувача за унікальним нікнеймом;
- обмін текстовими повідомленнями в реальному часі через глобальний чат та приватні діалоги;
- множинне закріплення повідомлень у діалогах із можливістю циклічної навігації хронологічною стопкою закрепу;
- закріплення та відкріплення пріоритетних чатів у лівій панелі з можливістю їх ручного сортування методом Drag-and-Drop;

- вимкнення звукових і візуальних сповіщень (мут) для окремих чатів зі зміною кольорової індикації;
- синхронне повне очищення історії приватного чату (видалення даних на клієнтських застосунках обох співрозмовників);
- комбінований пошук у реальному часі за іменами користувачів та текстом наявної історії повідомлень;
- локальна асинхронна архівація сесії (історії листування) у текстовий файл формату .txt із відображенням прогресу та можливістю переходу до каталогу збереження.

2.3 Характеристики користувачів

Цільовою аудиторією системи є співробітники невеликих компаній, студентські проєктні групи, організації з підвищеними вимогами до конфіденційності внутрішньої комунікації, а також користувачі локальних комп'ютерних мереж. Від користувача клієнтського застосунку не вимагається технічних знань – достатньо базових навичок роботи з операційною системою Windows, вміння користуватися мишею (зокрема для операцій Drag-and-Drop) та клавіатурою. У системі передбачено дві основні ролі з технічної точки зору: – користувач-клієнт – підключається до системи, веде листування, керує власним контентом та локальними архівами; – адміністратор сервера – технічний спеціаліст, що відповідає за запуск серверного застосунку, моніторинг виділення портів та підтримання працездатності локальної мережі.

2.4 Загальні обмеження

Робота десктопного клієнтського застосунку можлива виключно під управлінням операційної системи сімейства Windows, оскільки графічний інтерфейс побудовано з використанням технології WPF (Windows Presentation Foundation). Для коректної взаємодії між клієнтами та сервером необхідна наявність стабільного підключення до локальної мережі (LAN) або віртуальної приватної мережі (VPN) із підтримкою протоколу TCP/IP. Система орієнтована

Специфікація Програмного Забезпечення

виключно на обмін текстом: передача файлів, зображень, аудіо- та відеодзвінки не підтримуються. Оскільки система (в поточній версії) не використовує зовнішню базу даних, сервер зберігає дані лише в оперативній пам'яті. Це означає, що при перезавантаженні серверного застосунку загальна історія сесії скидається, що робить функцію локальної архівації на стороні клієнта критично важливою для збереження даних.

2.5 Припущення й залежності

Система припускає, що серверна частина запущена і доступна для прослуховування за вказаною IP-адресою та портом до того, як клієнтські застосунки спробують встановити з'єднання. Для запуску розробленого програмного забезпечення (як клієнта, так і сервера) на комп'ютерах користувачів має бути встановлене середовище виконання .NET (відповідної версії, наприклад .NET 8.0 Runtime). Механізм синхронного очищення чатів залежить від безперервності TCP-з'єднання: якщо один із клієнтів перебуває офлайн у момент ініціації команди на очищення іншим користувачем, пакет може бути не доставлений (до моменту реалізації черги офлайн-повідомлень на сервері). Якість відображення анімацій та швидкість рендерингу великих списків чатів залежить від наявності базового апаратного прискорення (DirectX) у системі користувача.

3 КОНКРЕТНІ ВИМОГИ

3.1 Вимоги до зовнішніх інтерфейсів

3.1.1 Інтерфейс користувача

Інтерфейс системи є класичним десктопним застосунком (Desktop Application) для операційної системи Windows, побудованим на технології WPF. Взаємодія з користувачем розпочинається зі стартового вікна авторизації, де користувач вводить свій унікальний нікнейм (до 15 символів) та IP-адресу локального сервера для підключення.

Після успішного підключення відкривається головне вікно чату, яке візуально розділене на дві основні робочі зони:

- ліва панель (список чатів) – містить рядок комбінованого пошуку та динамічний список користувачів/діалогів. Підтримує контекстне меню (ПКМ) для закріплення, вимкнення звуку та очищення історії. Інтерфейс підтримує перетаскування (Drag-and-Drop) закріплених чатів мишею для зміни їхнього пріоритету;
- права панель (зона повідомлень) – містить заголовок поточного чату, панель закріплених повідомлень (яка динамічно змінює стан при натисканні, перегортаючи хронологічну стопку закрепу), вікно виведення повідомлень та поле вводу тексту з кнопкою відправлення.

При натисканні на кнопку закриття вікна (хрестик) система перехоплює подію та викликає кастомне діалогове вікно виходу, де користувачу пропонується архівація сесії. Процес супроводжується візуальним індикатором завантаження (ProgressBar) та кнопкою швидкого переходу до створеного файлу.

3.1.2 Апаратний інтерфейс

Система взаємодіє з апаратним забезпеченням через стандартні підсистеми введення-виведення ОС Windows. Увесь ввід здійснюється через клавіатуру та

комп'ютерну мишу (наявність миші є критично важливою для виконання операцій Drag-and-Drop). Для забезпечення мережевої взаємодії використовується стандартний мережевий адаптер (Ethernet або Wi-Fi) пристрою користувача.

3.1.3 Програмний інтерфейс

Зовнішній програмний інтерфейс (комунікація між клієнтом та сервером) реалізований на базі низькорівельних мережевих сокетів (System.Net.Sockets.TcpClient та TcpListener). Обмін даними відбувається шляхом передачі серіалізованих об'єктів у форматі JSON. Кожен пакет даних (MessagePacket) містить тип команди (PacketType), ім'я відправника, цільового отримувача, текстовий вміст, часову мітку та унікальний ідентифікатор повідомлення. Сервер виконує роль маршрутизатора: приймає потік байтів, десеріалізує JSON, визначає отримувача та пересилає пакет у відповідний мережевий потік (NetworkStream) цільового клієнта.

3.1.4 Комунікаційний протокол

Уся взаємодія між клієнтами та сервером відбувається через транспортний протокол TCP/IP, що гарантує цілісність та правильну послідовність доставлення повідомлень. Дані кодуються у форматі UTF-8. Розділення пакетів у безперервному потоці TCP реалізовано за допомогою символу розриву рядка (\n), що дозволяє серверу та клієнтам коректно зчитувати окремі JSON-об'єкти.

3.1.5 Обмеження пам'яті

Вимоги до пам'яті залежать від ролі застосунку:

- клієнтський застосунок (WPF) потребує близько 100-150 МБ оперативної пам'яті для рендерингу графічного інтерфейсу та підтримки фонових мережевих потоків;
- серверний застосунок акумулює історію листування всіх активних сесій в оперативній пам'яті (in-memory). Обсяг спожитої пам'яті прямо

пропорційний кількості користувачів та обсягу переданих повідомлень протягом часу роботи сервера. Рекомендований обсяг ОЗП для сервера локальної мережі – від 2 ГБ.

3.1.6 Операції

Система підтримує такі основні операції:

- встановлення TCP-з'єднання та реєстрація нікнейму на сервері;
- відправлення, отримання та локальне видалення повідомлень;
- закріплення та відкріплення чатів (Pin/Unpin);
- зміна пріоритету закріплених чатів (Drag-and-Drop);
- множинне закріплення повідомлень у поточній бесіді;
- увімкнення/вимкнення сповіщень (Mute);
- відправка маскованої команди на синхронне повне очищення чату ([CMD_CLEAR_CHAT]);
- асинхронний запис об'єктів колекції повідомлень у текстовий файл.

3.1.7 Функції продукту

Функції клієнтського застосунку MTMessenger:

- підключення до сервера за введеною IP-адресою;
- ведення комунікації у загальному (глобальному) чаті або у приватних бесідах;
- миттєва фільтрація (пошук) списку чатів за іменами користувачів та вмістом повідомлень;
- виклик контекстного меню (ПКМ) на елементах списку чатів для управління їх станом;
- візуальне відстеження кількості непрочитаних повідомлень за допомогою лічильників;

- збереження історії сесії (експорт) у файл формату .txt при закритті програми із підстановкою нікнейму для наступного входу.

3.1.8 Припущення й залежності

Для розгортання та роботи системи припускається, що серверний застосунок запускається першим, і його IP-адреса та порт (наприклад, 127.0.0.1:8888 для локального тестування) відомі користувачам-клієнтам.

Для запуску скомпільованих файлів припускається наявність встановленого середовища виконання .NET Runtime (версії 8.0 або вище) на комп'ютерах користувачів. Оскільки система не використовує централізовану базу даних для довгострокового зберігання, припускається, що користувачі самостійно несуть відповідальність за збереження своїх даних шляхом використання вбудованої функції архівації при виході з програми.

3.2 Властивості програмного продукту

MTMessenger є багатопотоковою десктопною програмою, що працює в режимі реального часу. Система базується на реактивній моделі оновлення інтерфейсу (MVVM-подібний підхід із INotifyPropertyChanged), що дозволяє миттєво відображати зміни у станах чатів (поява нових повідомлень, зміна кольору лічильника при муті, поява іконки закріплення) без прямого перемальовування всього вікна.

Опрацювання вхідних мережевих пакетів відбувається в окремому асинхронному фоновому потоці (через Task.Run), а передача команд на оновлення графічних елементів безпечно делегується у головний потік інтерфейсу за допомогою об'єкта Dispatcher. Відмінною властивістю продукту є його повна незалежність від сторонніх хмарних сховищ, що гарантує високий рівень приватності у локальних корпоративних або домашніх мережах.

3.3 Атрибути програмного продукту

3.3.1 Надійність

Усі операції читання та запису в мережевий потік (NetworkStream) загорнуті у блоки обробки винятків (try-catch). У разі раптового розриву з'єднання або падіння сервера, клієнтський застосунок коректно перехоплює помилку (наприклад, InvalidOperationException), виводить інформаційне повідомлення користувачеві та не допускає аварійного завершення (крашу) програми.

3.3.2 Доступність

Доступність системи повністю залежить від доступності локальної мережі (LAN) та часу безперебійної роботи комп'ютера, на якому запущено серверний застосунок. За умови стабільної роботи локального комутатора, система забезпечує 100% доступність (uptime) для клієнтів.

3.3.3 Безпека

Безпека листування забезпечується фізичною або логічною ізоляцією локальної мережі. Дані між клієнтом і сервером передаються у відкритому вигляді (JSON), проте вони не виходять за межі корпоративного контуру. Дані сесії зберігаються виключно в оперативній пам'яті сервера та знищуються при його зупинці. Експортовані архіви зберігаються локально на жорсткому диску користувача.

3.3.4 Супроводжуваність

Код клієнтського застосунку чітко структурований: логіка роботи з моделлю даних чату винесена в окремий клас ChatListItem, структури мережевих повідомлень уніфіковані в MessagePacket, а візуальна частина відокремлена у файли розмітки XAML. Це дозволяє легко додавати нові типи пакетів (наприклад, передачу файлів у майбутньому) без необхідності переписування ядра програми.

3.3.5 Переносимість

Клієнтська частина системи є жорстко прив'язаною до операційної системи Windows (через використання фреймворку WPF). Серверна частина, написана на чистому C# (.NET Core/.NET 8), є кросплатформною і за потреби може бути скомпільована та запущена на серверах під управлінням Linux або macOS.

3.3.6 Продуктивність

Висока продуктивність графічного інтерфейсу під час роботи з великими обсягами історії повідомлень забезпечується вбудованими механізмами віртуалізації елементів списку (UI Virtualization) у WPF ListBox. Пошукова фільтрація реалізована через CollectionViewSource, що дозволяє фільтрувати колекції об'єктів у пам'яті клієнта практично миттєво без додаткових запитів до сервера.

3.4 Вимоги до збереження даних (Бази даних)

Архітектура програмної системи MTMessenger принципово відрізняється від традиційних месенджерів тим, що не використовує реляційні бази даних (SQL) на стороні сервера для довгострокового зберігання інформації. Збереження даних реалізується на двох незалежних рівнях:

- Оперативний рівень (In-Memory): Під час активної сесії вся історія повідомлень зберігається в оперативній пам'яті клієнта та сервера у вигляді колекцій словників (наприклад, Dictionary<string, List<ChatMessageModel>>). Ключем словника виступає ім'я співрозмовника, а значенням – хронологічний список повідомлень.
- Локальний персистентний рівень (Файлова система): Постійне зберігання інформації забезпечується виключно за бажанням самого користувача через механізм локальної архівації. За допомогою класу StreamWriter система

асинхронно серіалізує оперативний словник у структурований текстовий файл формату .txt з відповідними часовими мітками.

Такий підхід повністю унеможливлює витік історичних даних у разі компрометації сервера, оскільки сервер виступає виключно як транзитний вузол реального часу.

3.5 Інші вимоги

Усі матеріали комплексної кваліфікаційної роботи написані українською мовою.

Для розробки програмної системи використовуються такі засоби та технології:

- C# 12.0 / .NET 8.0 – основна мова програмування та платформа для розробки клієнтської і серверної частин;
- WPF (Windows Presentation Foundation) та XAML – технологічний стек для побудови графічного інтерфейсу користувача;
- System.Net.Sockets – стандартна бібліотека класів .NET для роботи з мережевими протоколами TCP/IP;
- System.Text.Json – бібліотека для швидкої серіалізації та десеріалізації об'єктів обміну даними;
- Visual Studio 2022 – інтегроване середовище розробки (IDE). Розробка та тестування проводились в операційній системі Windows 11.